

Risoluzione del Uncapacitated Facility Location Problem tramite metaeuristiche ILS e Simulated Annealing

Tommaso Bagiana,
Corso di Computational Intelligence

14 luglio 2026

Sommario

Il presente lavoro affronta il problema della localizzazione di impianti (*Facility Location Problem*), nella sua variante non capacitata (UFLP), tramite un approccio a due livelli: una metaeuristica esplora lo spazio delle configurazioni binarie di apertura/chiusura degli impianti, mentre il sottoproblema di trasporto associato a ciascuna configurazione viene risolto in modo esatto scegliendo la sede più vicina all'utente tra quelle aperte. Sono state implementate e confrontate due metaeuristiche, *Iterated Local Search* (ILS) e *Simulated Annealing* (SA), analizzandone i parametri, qualità delle soluzioni e costo computazionale su istanze di dimensione crescente. Il linguaggio scelto è stato OCaml per due caratteristiche principali. La prima è che il codice è particolarmente efficiente, la seconda è che OCaml è un linguaggio funzionale, il che permette di scrivere problemi matematici come la *Facility Location* in modo naturale e flessibile.

Indice

1	Definizione del problema	2
1.1	Uncapacitated Facility Location Problem (UFLP)	2
2	Dettagli implementativi	2
2.1	Struttura del progetto	2
2.2	Interfaccia a riga di comando	3
2.3	Raccolta dati	3
2.4	Guida rapida alla compilazione e all'uso	3
3	Approccio risolutivo	4
3.1	Implementazione della Funzione Obiettivo	4
4	Metaeuristiche	5
4.1	Iterated Local Search (ILS)	5
4.2	Simulated Annealing (SA)	6
5	Setup sperimentale	7
6	Risultati e analisi	7
6.1	Taratura dei parametri di Iterated Local Search	7
6.2	Taratura dei parametri di Simulated Annealing	8
6.3	Analisi delle curve di convergenza	9
7	Conclusioni	11

1 Definizione del problema

1.1 Uncapacitated Facility Location Problem (UFLP)

Dato un insieme di potenziali impianti $I = \{1, \dots, n\}$, ciascuno con costo di apertura f_i , e un insieme di clienti $J = \{1, \dots, m\}$, ciascuno con domanda d_j da soddisfare, e detto c_{ij} il costo di servire il cliente j dall'impianto i , la formulazione a programmazione lineare intera dell'UFLP è:

$$\min \sum_{i \in I} f_i x_i + \sum_{i \in I} \sum_{j \in J} c_{ij} d_j z_{ij} \quad (1)$$

$$\text{s.t.} \quad \sum_{i \in I} z_{ij} = 1 \quad \forall j \in J \quad (2)$$

$$\sum_{j \in J} z_{ij} \leq m x_i \quad \forall i \in I \quad (3)$$

$$x_i \in \{0, 1\} \quad \forall i \in I \quad (4)$$

$$z_{ij} \in \{0, 1\} \quad \forall i \in I, j \in J \quad (5)$$

dove x_i indica se l'impianto i è aperto e z_{ij} è 1 se la facility i fornisce il cliente j , 0 altrimenti. Il vincolo (2) impone che ogni domanda sia interamente soddisfatta, mentre il vincolo (3) impedisce di servire clienti da impianti chiusi.

2 Dettagli implementativi

2.1 Struttura del progetto

Il progetto è strutturato in un approccio ibrido: il motore algoritmico è implementato in OCaml. La scelta è ricaduta su questo linguaggio per massimizzare le prestazioni, inoltre ocaml è un linguaggio funzionale, caratteristica che lo rende flessibile adatto a problemi matematici. L'orchestrazione degli esperimenti e l'analisi dei risultati sono gestite tramite script Python.

La directory principale del progetto espone la seguente gerarchia:

- **main_program/**: È un progetto gestito con Dune che contiene l'implementazione in OCaml del risolutore.
 - **lib/**: Contiene i moduli della libreria. Troviamo **objective_function.ml** per la valutazione delle soluzioni, **ils.ml** e **sa.ml** per l'esecuzione delle metaeuristiche, **reader.ml** per il parsing delle istanze da file testuali e **csv_logger.ml** per tracciare e salvare i risultati.
 - **bin/main.ml**: Contiene l'interfaccia a riga di comando (CLI) e funge da *entry point* dell'eseguibile.
- **Script Python** (**tuning.py**, **analyze_tuning.py**, **plot_traces.py**): Situati nella directory radice, questi file si occupano di avviare sistematicamente l'eseguibile OCaml passandogli i parametri, calcolare l'analisi di sensibilità sugli iperparametri e processare i tracciati di convergenza.
- **Directory grafiche e di output** (**figs/**, **analysis_output/**): Raccogliono le visualizzazioni generate dagli script Python, impiegate all'interno di questa relazione per l'esposizione dei risultati.

2.2 Interfaccia a riga di comando

Il programma espone un'interfaccia a riga di comando che consente di selezionare l'istanza da risolvere, l'algoritmo (`ils` o `sa`) e i relativi iperparametri. Il flag `-notrace` consente di disattivare il logging per-iterazione (utile per ridurre l'overhead durante le campagne di tuning su larga scala), mantenendo comunque attivo il logging aggregato su `runs.csv`.

2.3 Raccolta dati

Ogni esecuzione produce una riga in `runs.csv` con il costo della soluzione trovata, il tempo di esecuzione, l'iterazione in cui è stata trovata la soluzione migliore e la configurazione di parametri utilizzata. Quando il logging per-iterazione è attivo, viene inoltre prodotto un file `trace.csv` per ciascuna esecuzione, contenente il costo della soluzione corrente e della migliore soluzione trovata ad ogni iterazione, utilizzato per le analisi di convergenza.

2.4 Guida rapida alla compilazione e all'uso

Compilazione ed esecuzione del solver OCaml. Il progetto `main_program/` è gestito con Dune. Per compilare:

```
cd main_program
dune build
```

Il programma può quindi essere eseguito tramite `dune exec`, specificando l'istanza, l'algoritmo e i relativi iperparametri, ad esempio:

```
dune exec bin/main.exe -- -algo sa -instance <path_istanza> \
-t0 100 -t_end 0.1 -alpha 0.9 -new_low 50 -seed 111
```

Il flag `-notrace` disattiva il logging per-iterazione (utile nelle campagne di tuning su larga scala), mantenendo comunque attivo il logging aggregato su `runs.csv`.

Script Python. Gli esperimenti vengono orchestrati ed analizzati tramite tre script, eseguiti nell'ordine seguente.

1. `tuning.py`: lancia sistematicamente l'eseguibile OCaml sulla griglia di iperparametri (Tabella 1), producendo `runs.csv` e, se il logging non è disattivato, i file `trace_*.csv`.
2. `analyze_tuning.py`: elabora `runs.csv` e produce l'analisi di sensitivity (Sezioni 6):

```
python3 analyze_tuning.py runs.csv figs
```

Genera `ils_sensitivity.png`, `sa_sensitivity.png`, un grafico separato per ciascun parametro SA con effetto non trascurabile (es. `sa_sensitivity_new_low.png`), `quality_vs_time.png` e `best_configs_summary.csv`. Lo script seleziona automaticamente, per ciascuna dimensione n , se includere i dati nelle medie aggregate in base alla completezza della copertura sperimentale (si veda la nota alla Sezione 6.2); le esclusioni vengono stampate a schermo durante l'esecuzione.

3. `plot_traces.py`: elabora i file `trace_*.csv` e produce le curve di convergenza (Figura 4):

```
python3 plot_traces.py --trace-dir ./traces --runs-csv runs.csv \
--outdir figs_traces
```

Genera `master_trace_long.csv` (tabella unica con tutte le traiettorie) e, per ogni n trovato, `convergence_n<N>.png` e `convergence_n<N>_<algo>_variance.png`. Elabora i file uno alla volta e salva i risultati progressivamente, così un'interruzione a metà non fa perdere il lavoro già fatto: rilanciando lo stesso comando, i file già elaborati vengono saltati automaticamente. Altri flag utili: `--no-resume` (riparte da zero), `--no-collapse` (disattiva la compressione delle righe a costo invariato) e `--skip-plots` (elabora solo i dati, utile con moltissimi file di trace, rimandando la generazione dei grafici a una chiamata successiva).

3 Approccio risolutivo

3.1 Implementazione della Funzione Obiettivo

Per valutare il costo totale di una determinata configurazione binaria x di apertura degli impianti, è stata implementata una funzione obiettivo che calcola la somma dei costi fissi di apertura e dei costi di trasporto ottimi (associati al rispettivo sottoproblema).

Di seguito viene riportato l'estratto del codice OCaml che si occupa di strutturare questa valutazione per il caso non capacitato (UFLP), isolando la matrice dei costi validi rispetto agli impianti attivi ed effettuando il calcolo posizionale del costo minimo per ciascun cliente:

Listing 1: Calcolo della funzione obiettivo per l'UFLP in OCaml.

```

1  (* prende istanza e restituisce funzione obiettivo corrispondente *)
2  let objective_function instance x =
3      match instance.u with
4      | None ->
5          let correct_cost_mat =
6              Array.map2
7                  (fun cost_row is_open ->
8                      if is_open then cost_row
9                      else Array.make (Array.length cost_row) max_int)
10                 instance.c x
11          in
12          let rec aux j acc =
13              if j < instance.m then
14                  let costo_spedizione =
15                      Array.fold_left (fun best row -> min best row.(j)) max_int
16                                     correct_cost_mat
17                  in
18                  if costo_spedizione = max_int then
19                      Infeasible
20                  else
21                      aux (j + 1) (costo_spedizione * instance.d.(j) + acc)
22              else
23                  Finite acc
24          in
25          aux 0 (opening_cost x instance.f) (* sommo costo spedizioni al
                                             costo di apertura *)

```

Nel frammento di codice sopra riportato (Listato 1), si nota l'uso del tipo algebrico per gestire l'ammissibilità della soluzione (`Finite` o `Infeasible`) e il pattern matching per intercettare l'eventuale presenza di capacità (`Some u`).

4 Metaeuristiche

4.1 Iterated Local Search (ILS)

L'ILS implementato opera sul vettore binario x di apertura/chiusura degli impianti, alternando fasi di ricerca locale e fasi di perturbazione:

- **Perturbazione:** applicata tramite *bit-flip* probabilistico su ciascuna componente di x , con probabilità di mutazione `p_mut` configurabile.
- **Criterio di arresto:** numero massimo di iterazioni `max_iterations`.

Listing 2: Local search e iterated search in OCaml.

```
1  (* TODO: l'operatore << qui sotto non e' standard OCaml (il confronto e'
   <).
2  Se e' un operatore custom definito altrove per confrontare valori di
   tipo
3  Finite/Infeasible, aggiungere una riga di testo che lo spieghi (
   altrimenti
4  il listato sembra non compilare); se e' un refuso di trascrizione,
5  sostituirlo con la funzione di confronto realmente usata in ils.ml.
   *)
6  let best_neighbor (x0, f_x0) f = (* f: objective function *)
7    let neighbors    = neighbors_of x0 in
8    let f_neighbors  = List.map f neighbors in
9    let f_graph      = List.combine neighbors f_neighbors in
10   let rec aux best = function
11     [] -> best
12     | (x, fx) :: rest ->
13       if fx << snd best then aux (x, fx) rest
14       else aux best rest
15   in aux (x0, f_x0) f_graph
16
17  let rec local_search (x0, f_x0) f = (* passo coppia invece che solo x0
   per evitare ricalcoli *)
18   let (best, f_best) = best_neighbor (x0, f_x0) f in
19   if f_best << f_x0 then
20     local_search (best, f_best) f
21   else
22     (x0, f_x0)
23
24  let iterated_search ~rng ~log ~iterations ~p_mut ~instance ~f =
25   let x0 = Array.init instance.n (fun _ -> Random.State.bool rng) in
26   let (x0, f_x0) = local_search (x0, f x0) f in
27   let rec aux current f_current best_iter i =
28     (match log with
29      | Some file ->
30        Csv_logger.trace_logger ~file ~iteration:i
31          ~current_cost:f_current ~best_cost:f_current
32      | None -> ());
33   if i < iterations then
34     let tmp = perturbation rng current p_mut in
35     let (next, f_next) = local_search (tmp, f tmp) f in
36     if f_next << f_current then
37       aux next f_next (i + 1) (i + 1)
38     else
39       aux current f_current best_iter (i + 1)
```

```

40     else
41         (current, f_current, best_iter, iterations)
42 in
43 aux x0 f_x0 0 0

```

4.2 Simulated Annealing (SA)

Il SA implementato adotta uno schema di raffreddamento, controllato dai seguenti parametri:

- t_0 : temperatura iniziale;
- t_{end} : temperatura finale (criterio di arresto);
- α : fattore di raffreddamento geometrico, $t_{k+1} = \alpha \cdot t_k$;
- **new_low**: lunghezza della catena di Markov (numero di iterazioni eseguite a temperatura costante prima di raffreddare).

Ad ogni iterazione viene generata una soluzione vicina tramite bit-flip di una componente di x ; la mossa viene accettata se migliorativa, oppure con probabilità $e^{-\Delta/t}$ se peggiorativa. La funzione restituisce la *migliore* soluzione incontrata durante l'intera esecuzione, e non semplicemente lo stato finale della catena.

Listing 3: Codice principale del Simulated Annealing.

```

1  (* TODO: come sopra, verificare "<" (confronto) e "%" qui sotto (in
   OCaml
2  il modulo e' l'operatore/keyword "mod", non "%") rispetto al codice
3  realmente compilato in sa.ml: se sono operatori custom vanno spiegati
   ,
4  altrimenti vanno corretti nel listato. *)
5  let accept_solution rng t f_old f_new = (* restituisce true se sceglie
   nuova soluzione *)
6  match f_new, f_old with
7  | Infeasible, _ ->
8      false (* non accettiamo mai una mossa che porta a una soluzione
   infeasible *)
9  | Finite _, Infeasible ->
10     true (* qualunque soluzione feasible e' sempre meglio di una
   infeasible *)
11 | Finite new_c, Finite old_c ->
12     let delta = new_c - old_c in
13     if delta <= 0 then true
14     else
15         let p = exp (-(float_of_int delta) /. t) in
16         (Random.State.float rng 1.0) < p
17
18
19 let simulated_annealing ~log ~rng ~t0 ~t_end ~alpha ~new_low ~instance ~
   f =
20 let s0 = Array.init instance.n (fun _ -> Random.State.bool rng) in
21 let f_s0 = f s0 in
22 let rec aux i (s, f_s) (best, f_best) best_iter t =
23     (match log with
24     | Some file ->
25         Csv_logger.trace_logger ~file ~iteration:i
26         ~current_cost:f_s ~best_cost:f_best
27     | None -> ());

```

```

28   if t < t_end then
29       (best, f_best, best_iter, i - 1)
30   else
31       let next = random_neighbor rng s instance.n in
32       let f_next = f next in
33       let accepted = accept_solution rng t f_s f_next in
34       let best, f_best, best_iter =
35           if f_next << f_best then (next, f_next, i) else (best, f_best,
36               best_iter)
37       in
38       let t = if i %% new_low = 0 then alpha *. t else t in
39       if accepted then
40           aux (i + 1) (next, f_next) (best, f_best) best_iter t
41       else
42           aux (i + 1) (s, f_s) (best, f_best) best_iter t
43   in
44   aux 1 (s0, f_s0) (s0, f_s0) 0 t0

```

5 Setup sperimentale

Gli esperimenti sono stati condotti su istanze di dimensione crescente ($n \in \{5, 10, 20, 50, 100\}$, con $m = 2n$), ciascuna risolta con tre seed distinti (111, 222, 333).

Per ciascun algoritmo è stata condotta una campagna di tuning sui seguenti iperparametri:

Algoritmo	Parametri esplorati
ILS	$\text{max_iterations} \in \{10, 100, 1000, 10000\}$ $\text{p_mut} \in \{0.01, 0.05, 0.1\}$
SA	$t_0 \in \{10, 50, 100, 300, 600\}$ $\alpha \in \{0.85, 0.90, 0.95, 0.99\}$ $\text{new_low} \in \{10, 50, 100\}$

Tabella 1: Griglia di iperparametri esplorata nella fase di tuning.

6 Risultati e analisi

6.1 Taratura dei parametri di Iterated Local Search

Per individuare una configurazione efficace dell'algoritmo ILS è stata effettuata un'analisi di sensibilità sui principali parametri, variando il numero massimo di iterazioni (max_iterations) e la probabilità di perturbazione (p_mut).

La Figura 1 riporta, rispettivamente, la frazione di esecuzioni che raggiungono la migliore soluzione osservata e il tempo medio di esecuzione al variare dei parametri.

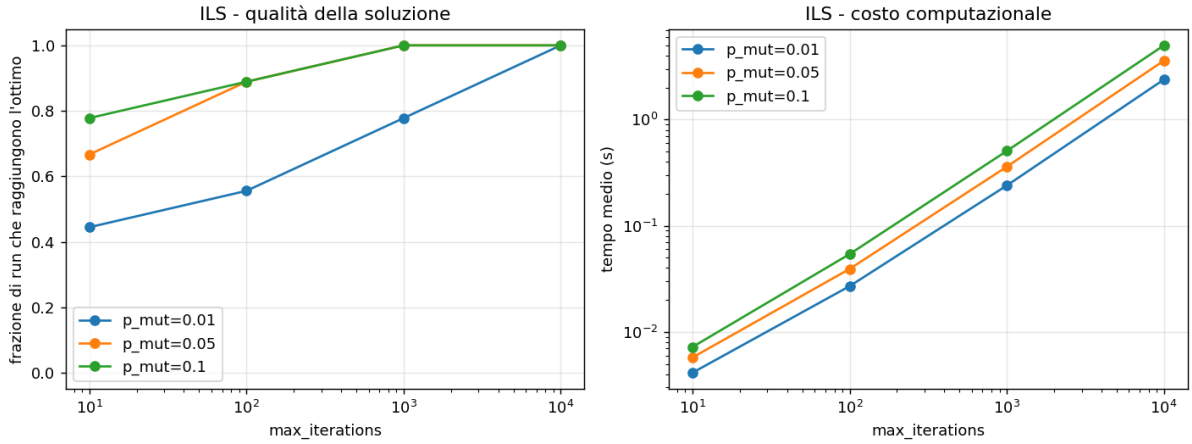


Figura 1: Analisi di sensibilità dei parametri di Iterated Local Search. A sinistra è riportata la qualità della soluzione, espressa come frazione di esecuzioni che raggiungono la migliore soluzione osservata; a destra il tempo medio di esecuzione.

Dal grafico relativo alla qualità della soluzione emerge che l'aumento del numero massimo di iterazioni migliora progressivamente le prestazioni dell'algoritmo. Tuttavia, oltre circa 1000 iterazioni il beneficio diventa trascurabile, poiché tutte le configurazioni raggiungono la migliore soluzione nella quasi totalità delle esecuzioni.

L'influenza della probabilità di perturbazione risulta invece più evidente per un numero ridotto di iterazioni. Il valore $p_mut = 0.01$ mostra una capacità di esplorazione inferiore, raggiungendo con minore frequenza la soluzione migliore. Al contrario, i valori $p_mut = 0.05$ e $p_mut = 0.1$ permettono di ottenere prestazioni significativamente migliori già con poche centinaia di iterazioni.

Il secondo grafico evidenzia il costo computazionale associato alle diverse configurazioni. Come prevedibile, il tempo di esecuzione aumenta all'aumentare del numero massimo di iterazioni. Inoltre, il valore $p_mut = 0.01$ richiede tempi sensibilmente maggiori per raggiungere prestazioni confrontabili con le altre configurazioni, mentre $p_mut = 0.05$ e $p_mut = 0.1$ mantengono un costo computazionale inferiore.

Considerando congiuntamente qualità della soluzione e tempo di esecuzione, la configurazione con $p_mut = 0.05$ e $max_iterations = 100$ rappresenta il miglior compromesso tra efficacia ed efficienza. Tale configurazione è stata pertanto adottata nelle successive prove di confronto e nell'analisi di scalabilità.

6.2 Taratura dei parametri di Simulated Annealing

Per capire l'ordine di importanza dei parametri di SA ho fissato un parametro alla volta ed ho calcolato la media della *hit-rate* (percentuale di run che ha raggiunto il costo migliore).

Parametro	Ampiezza effetto su hit-rate
t_0	0.354
α	0.174
new_low	0.112
t_{end}	0.004

Tabella 2: Ampiezza dell'effetto marginale di ciascun iperparametro SA sulla frazione di run che raggiungono il costo ottimo.

Il risultato più rilevante è che t_0 ha un impatto maggiore di α , contro l'aspettativa iniziale che avrebbe suggerito di concentrare l'analisi sulla coppia $(\alpha, \text{new_low})$ in quanto concettualmente più legata al comportamento dell'algoritmo (velocità di raffreddamento e ampiezza dell'esplorazione a ciascuna temperatura). E' possibile che ciò sia causato dal fatto che ci siano più opzioni per t_0 e meno per new_low .

La Figura 2 mostra quindi l'interazione tra t_0 e α , i due parametri con maggiore impatto.

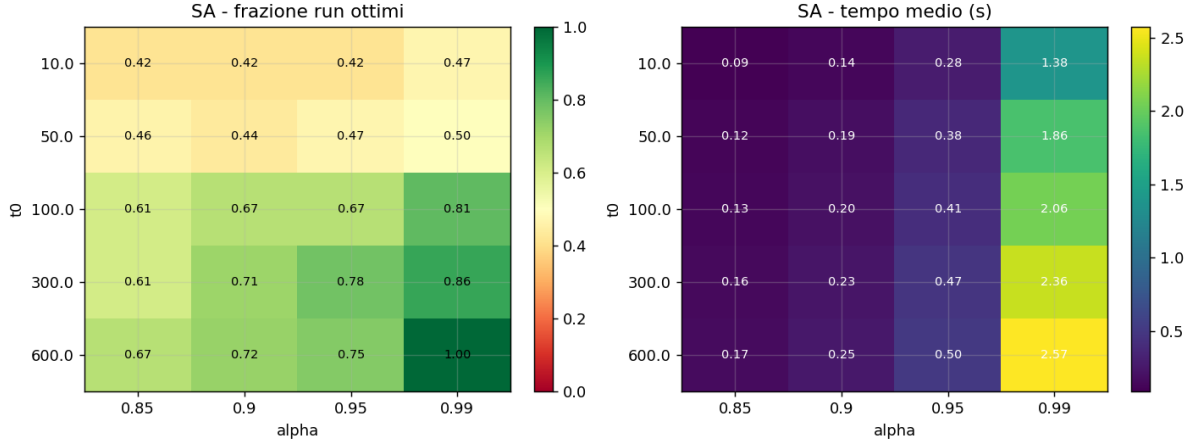


Figura 2: SA — frazione di run che raggiungono l'ottimo e tempo medio, al variare di t_0 e α ($n \in \{5, 10, 20, 50\}$).

La hit-rate cresce in modo pressoché monotono lungo entrambi gli assi, dal minimo di 0.42 ($t_0 = 10$, $\alpha = 0.85$) al massimo di 1.00 ($t_0 = 600$, $\alpha = 0.99$): i due parametri hanno un effetto che si rinforza a vicenda. Il tempo medio cresce di pari passo, da 0.09s a 2.57s tra gli stessi due estremi, con un'eccezione: la colonna $\alpha = 0.99$ è nettamente la più costosa in tempo (fino a 2.57s), più di quanto lo sia l'ultima riga $t_0 = 600$ (fino a 1.38s a $\alpha = 0.85$). A parità di guadagno in hit-rate, alzare t_0 sembra più "efficiente" di alzare α .

Il parametro restante non trascurabile, new_low , viene mostrato separatamente in Figura 3, marginalizzando su tutti gli altri parametri.

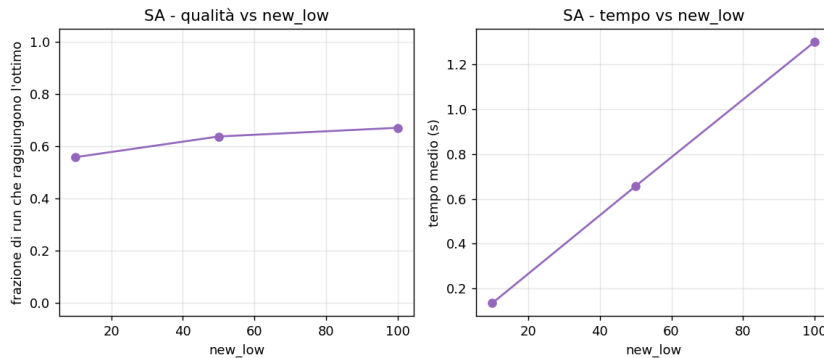


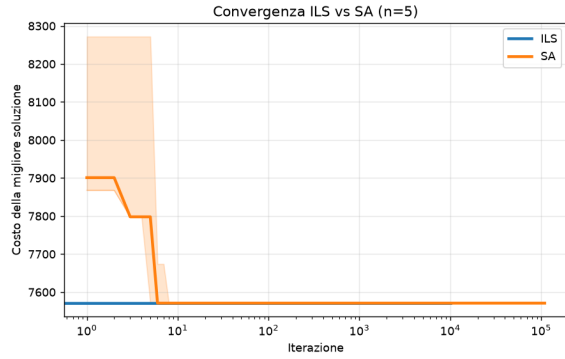
Figura 3: SA — frazione di run che raggiungono l'ottimo e tempo medio, al variare di new_low .

6.3 Analisi delle curve di convergenza

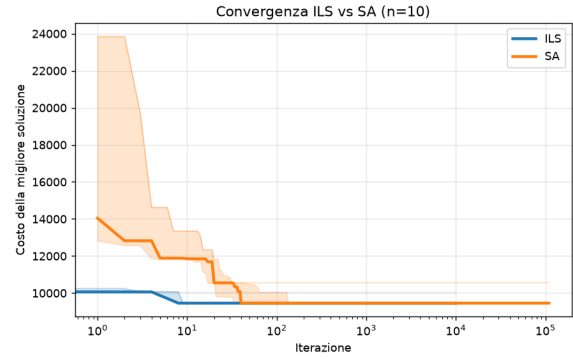
Per comprendere meglio il comportamento dinamico dei due algoritmi sono state analizzate le curve di convergenza del costo della migliore soluzione in funzione delle iterazioni. Per ciascuna dimensione del problema ($n = \{5, 10, 20, 50\}$) sono stati eseguiti tre run indipendenti con

seed differenti; nei grafici viene riportata la media delle traiettorie, mentre l'area ombreggiata rappresenta una deviazione standard.

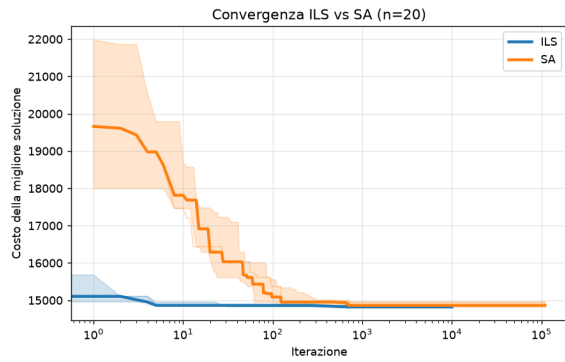
Le Figure dalla 4a alla 4e mostrano il confronto tra Iterated Local Search (ILS) e Simulated Annealing (SA) al variare della dimensione dell'istanza.



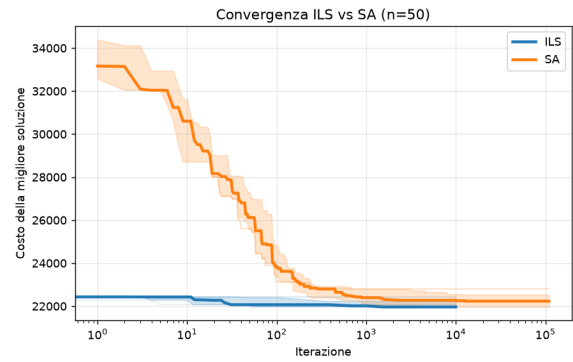
(a) $n = 5$



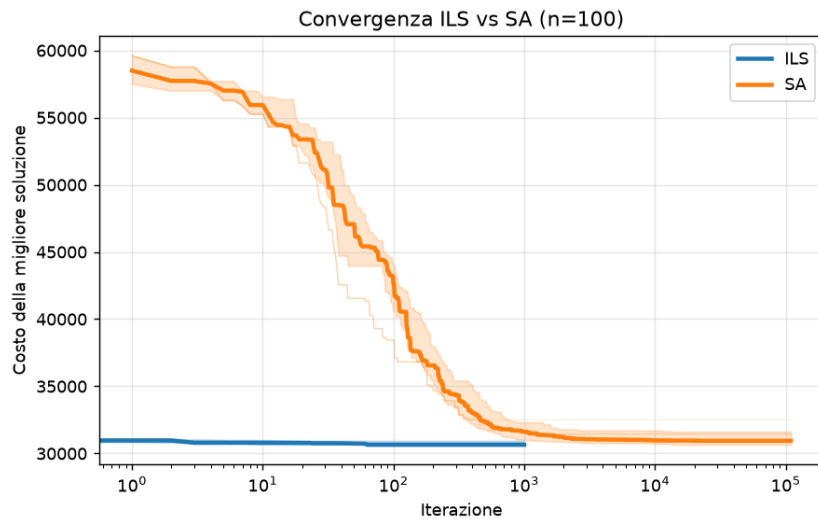
(b) $n = 10$



(c) $n = 20$



(d) $n = 50$



(e) $n = 100$

Figura 4: Confronto delle curve di convergenza di ILS e SA per le diverse dimensioni del problema. Le linee rappresentano la media di tre esecuzioni indipendenti, mentre l'area ombreggiata indica ± 1 deviazione standard.

I risultati confermano le differenze già evidenziate dall'analisi quantitativa precedente.

L'Iterated Local Search mostra una convergenza estremamente rapida. In tutti gli esperimenti la migliore soluzione viene individuata nelle primissime iterazioni e successivamente il costo rimane pressoché costante. La banda di deviazione standard è inoltre molto ridotta, indicando che l'algoritmo produce risultati altamente riproducibili indipendentemente dal seed iniziale.

Al contrario, Simulated Annealing presenta un comportamento molto più graduale. Il costo diminuisce progressivamente durante il processo di raffreddamento, con miglioramenti distribuiti lungo un numero elevato di iterazioni. La maggiore ampiezza della banda di deviazione standard nelle prime fasi evidenzia una significativa variabilità tra le diverse esecuzioni, dovuta alla natura probabilistica dell'algoritmo. Tale variabilità tende però a ridursi man mano che la temperatura diminuisce e le diverse esecuzioni convergono verso soluzioni molto simili.

Per l'istanza più piccola ($n = 5$) entrambi gli algoritmi raggiungono rapidamente la stessa soluzione finale. In questo caso SA necessita di poche iterazioni aggiuntive rispetto a ILS, ma la differenza risulta trascurabile.

Per $n = 10$ emerge una differenza più evidente: ILS raggiunge quasi immediatamente un buon minimo locale, mentre SA continua a migliorare la soluzione fino a circa alcune decine di iterazioni, raggiungendo infine un costo leggermente inferiore. La maggiore dispersione iniziale delle traiettorie di SA indica una forte dipendenza dalle mosse casuali effettuate durante le prime fasi dell'esplorazione.

Anche per $n = 20$ il comportamento osservato rimane sostanzialmente invariato. ILS converge quasi istantaneamente e mostra una variabilità minima tra i diversi seed. Simulated Annealing continua invece a migliorare la soluzione in modo graduale, riducendo progressivamente anche la variabilità tra le esecuzioni. Dopo circa un centinaio di iterazioni le traiettorie risultano quasi completamente sovrapposte.

Infine, per l'istanza più grande ($n = 50$), la differenza tra i due algoritmi diventa ancora più marcata. ILS raggiunge rapidamente una soluzione stabile senza ulteriori miglioramenti significativi, mentre SA continua a esplorare lo spazio delle soluzioni per numerose iterazioni, ottenendo riduzioni del costo distribuite lungo tutto il processo di raffreddamento. Sebbene richieda un numero di iterazioni decisamente superiore, SA converge verso una soluzione finale sostanzialmente coincidente con quella ottenuta da ILS.

Nel complesso, queste curve confermano la diversa filosofia dei due metaeuristici. ILS privilegia uno sfruttamento intensivo dello spazio di ricerca, individuando rapidamente un buon minimo locale e mostrando un'elevata stabilità tra esecuzioni differenti. Simulated Annealing, invece, dedica una parte significativa dell'esecuzione all'esplorazione dello spazio delle soluzioni, accettando temporaneamente anche mosse peggiorative; ciò comporta una convergenza più lenta e una maggiore variabilità iniziale.

7 Conclusioni

Il presente lavoro ha affrontato la risoluzione dell'Uncapacitated Facility Location Problem tramite due metaeuristiche, Iterated Local Search e Simulated Annealing, costruite su una valutazione esatta della funzione obiettivo (assegnazione di ciascun cliente alla sede aperta più economica) e confrontate su istanze di dimensione crescente da $n = 5$ a $n = 100$.

Risultati principali. Entrambi gli algoritmi si sono dimostrati in grado di raggiungere il migliore costo noto sulla quasi totalità delle istanze testate, purché configurati con un budget di ricerca sufficiente. La differenza sostanziale non è tanto nella qualità finale raggiungibile, quanto nel *modo* in cui i due algoritmi vi arrivano, come emerso dalle curve di convergenza (Sezione 6): ILS individua una soluzione di ottima qualità entro poche decine di iterazioni e vi resta sostanzialmente stabile, con una variabilità tra seed minima; SA richiede invece alcuni ordini di grandezza di iterazioni in più, mostrando un miglioramento progressivo e una variabilità

iniziale significativamente più ampia, che si riduce mano a mano che la temperatura scende. Per le istanze più grandi ($n = 50$, $n = 100$) questa differenza si accentua ulteriormente: SA impiega centinaia o migliaia di iterazioni per avvicinarsi alla soluzione che ILS individua quasi immediatamente, pur convergendo infine verso un costo finale comparabile.

Cosa conta per ciascun algoritmo. L’analisi di sensitivity ha mostrato che i due algoritmi non sono ugualmente sensibili a tutti i loro iperparametri. Per ILS, sia `max_iterations` che `p_mut` incidono sulla probabilità di raggiungere l’ottimo, ma con budget di iterazioni ridotto (10–100) un `p_mut` troppo basso (0.01) penalizza sensibilmente la qualità rispetto a valori più esplorativi (0.05–0.1). Per SA, la temperatura iniziale t_0 si è rivelata il parametro più determinante (ampiezza dell’effetto marginale sulla hit-rate: 0.35, contro 0.17–0.22 di α e 0.11–0.15 di `new_low`), mentre la temperatura finale t_{end} non ha mostrato alcun effetto misurabile nel range testato (Tabella 2).

Limiti dello studio. Vanno segnalati alcuni limiti nella campagna sperimentale condotta. In primo luogo, il progetto copre esclusivamente la variante non capacitata del problema (UFLP); l’estensione al caso capacitato (CFLP) non è stata implementata. In secondo luogo, la campagna di tuning di ILS non copre in modo completo la combinazione di `max_iterations` alto (≥ 1000) sulle istanze più grandi ($n = 50$, $n = 100$): alcune run non sono state completate nei tempi a disposizione, e i risultati relativi a quella porzione della griglia (esclusi dalle medie riportate in Sezione 6) andrebbero considerati preliminari. Infine, ciascuna configurazione è stata valutata su soli 3 seed: sufficienti per individuare pattern generali netti, ma non per stime di affidabilità statisticamente strette, specialmente per le configurazioni con hit-rate intermedia (né vicina a 0 né a 1), dove la variabilità tra singole esecuzioni pesa di più.